Decomposing Kernel Resilience into Arithmetic and Structural Masking

An analytical companion to NVBit-FI

SOGA team (Emilio Incerto), note for Lishan Yang

2026-06

A proposed joint experiment around the SUGAR / Typhoon line of work.

---

1. What SOGA adds

SOGA (Second-Order Gaussian Approximation) is an exact symbolic inference engine. We encode a single-bit-flip fault model (the IEEE-754 sign / exponent / mantissa bit classes) into a floating-point kernel and propagate it analytically, obtaining the distribution of the output perturbation, hence MSK / SDC / OTR rates without sampling. Versus an NVBit-FI campaign (~10^3 injections per input), SOGA is deterministic where sampling is statistical, instant per input, and, crucially, can hold the input statistics fixed while varying one factor at a time.

2. The core idea, a decomposition neither tool gives alone

NVBit-FI measures total masking, which has two sources:

```
MSK_total (NVBit-FI)  =  MSK_structural  ⊕  MSK_arithmetic
                    └ fault never reaches  └ fault reaches the computation but is
                      a live value: dead/   absorbed by rounding / cancellation /
                    overwritten/predicated tolerance  →  SOGA computes this analytically
                        → hardware-specific     → algorithm + precision specific
```

SOGA computes `MSK_arithmetic` analytically. So the gap `MSK_total − MSK_arithmetic` isolates the structural (hardware) component, which no analytical model, and no FI campaign on its own, can separate. That is the joint contribution.

3. Why floating-point

SOGA is native on continuous values, so float is where it is most natural and where the physics is real:

- A mantissa-bit flip is a small relative perturbation; rounding absorbs (masks, bit-exactly) any perturbation below the ULP of the result. How much masking occurs depends on how the term magnitudes are distributed relative to the output, a spread the Frobenius norm does not capture. So resilience depends on the magnitude distribution inside each accumulation, not on the norm.
- On bit-exact integers this masking does not exist (magnitude is provably irrelevant), which is exactly why we target float.

- Fault model = the IEEE-754 bit classes (sign / exponent / mantissa); exponent flips yield SDC or OTR (overflow → Inf/NaN).

4. Minimal example, same norm, different masking (one float32 inner product)

One output cell `out = Σ x_k` (length 16, float32). Two inputs with identical Frobenius norm ‖x‖ = 4, each faulted by exact enumeration of every single-bit flip:

input (same ‖x‖F = 4)	output	MSK, bit-exact (ε = ULP)	MSK, ε = 5 %
Balanced (all 1)	16.0	12.5 %	78 %
Concentrated (3.9, then 0.23 × 15)	7.4	4.1 %	89 %

Same Frobenius norm, very different masking, and the gap even reverses sign with the criterion: bit-exact comparison favors the balanced input, a 5 % tolerance favors the concentrated one. So the norm does not predict resilience, the magnitude distribution does, in interaction with the correctness criterion. A norm sweep (e.g. Binomial in p) moves norm and distribution together and cannot separate them; SOGA holds the norm fixed and can. (Reproducible: *`float_example.py`*.)

5. Proposed joint experiment

- SOGA (us): compute the `MSK_arithmetic` baseline analytically for a kernel of your choice (`GEMM` / `2mm` / `MVT`) across input distributions, and generate discriminating inputs, same norm, large predicted MSK gap.
- NVBit-FI (you): measure `MSK_total` on the same inputs.
- Result: (i) decompose arithmetic vs structural masking; (ii) test whether the Frobenius-norm trend is arithmetic or structural; (iii) identify which input statistics actually drive resilience. Predicted analytically, confirmed on GPU.

6. Scope & honesty

- SOGA propagates real arithmetic; it approximates IEEE-754 rounding by masking perturbations below the output ULP (`ε = ULP`), and can add an application-level tolerance. It models the arithmetic component only, the structural part is precisely what the SOGA-vs-NVBit-FI gap reveals.
- Demonstrated on an integer kernel: SOGA reproduces a brute-force register-level single-bit-flip campaign exactly, outcome-by-outcome. Extending the exact propagation to the floating-point fault model, and validating against your NVBit-FI, is the proposed work.

7. One question to align our setups

Which target kernel, and is your correctness criterion bit-exact or application-tolerance based? That fixes ε and lets us match your campaign exactly.